

5. C vs OOP — 3 Problems Solved

C / Procedural Version	OOP Version
Data is stored separately in global arrays	Data and methods are grouped inside classes
Managing task status requires separate arrays	Each Task object has its own isDone status
As the program grows, many functions and global variables become difficult to manage	Classes organize the code and make it easier to maintain

Problem 1: Global Data In C:

```
char tasks[5][100];
```

```
int taskDone[5];
```

These are global variables.

Any function can potentially modify them.

In OOP:

```
class Task:
```

```
    def __init__(self, title):
```

```
        self.title = title
```

```
        self.isDone = False
```

The task's data and its functions are kept together.

This is called encapsulation.

Problem 2: Separate Data In C we used:

```
tasks[5]
```

```
taskDone[5]
```

We need two arrays to represent one task.

In OOP, one object contains everything:

```
task.title
```

```
task.isDone
```

For example:

Task Object

```
|
```

```
|-- title = "Study Python"
```

```
|-- isDone = False
```

Much easier to understand.

Problem 3: Code Becomes Difficult to Manage In a bigger C program, we may have:

addTask()

deleteTask()

markTaskDone()

showTasks()

updateTask()

and several global arrays.

In OOP, related functionality can stay inside:

class TaskList:

addTask()

markTaskDone()

showTasks()